Kandidatnr.: 48

A.1:

To reflect on the guest lectures by Chhagan Lal (NTNU) on resilience and smart manufacturing, and the lecture by Dr. Indika Balapuwaduge (UiA) on QoS mechanisms in MQTT. Together, these lectures gave me a more practical understanding of IoT security beyond what the course material gave me.

Chhagan Lal's lecture helped me understand how that the increasing connectivity in industrial enviroments through IoT sensors, cloud platforms and Ai increase the attack surface. Chhagan illustrated through a threat landscape diagram showing different types of attack vectors all from enterprice network down to PLC and field devices. This gave a more practical view on the course and made it seem like what we learned in the text book actually have real world applications.

The thing i found most valuable from the lecture is the emphasis Chhagan had on the convergence of IT and OT systems. Before this lecture i hadnt given much thought to how connecting isolated industrial systems to the internet creates entirely new categories of risk. Chhagan also introduced some frameworks such as IEC 62443 abd the NIST Cybersecurity Framework, which helped me understand why standard IT security approaches are not sufficient for industrial IoT environments. His discussion of resilience not just defens, but the ability to absorb and recover from disruptions expanded how i think of IoT secutiy as a whole.

Dr. Balapuwaduges lecture game me a deeper understanding of MQTT, which the course material had covered but at a fundamental level. He explained the 3 QoS levels in detail and how they represent a trade off between reliability and overhead.this was something i had given little thought to before this lecutre. How QoS levels directly affect how much damage denial of service can cause, in medical devices this can in worst case be fatal making QoS 2 not just a preference but a safety requirement.

Dr. Balapuwaduges also talked about the broker.centric nature of MQTT and what that means for security. Since all traffic flows through the broker it becomes a priority for attackers. The lecture talked about MQTTs weak default security where connections are unauthenticated by default this aligns well with that we know MQTT was not created with security in mind.

These two lecures, taken together, really made me think about IoT security in every layer, from the network architecture down to individual protocol design choices. Both lecures highlighted the gap between theoretical security and real world implementation, and that it is essential for me to understand this gap.

A.2:

Slide 12 of Chhagans lecture identified three major threas in smart manufacturing: ransomware, espionage, and supply chain attacks. This discussion focuses on supply chain attacks, as it is the most complex and difficult threat to defend against in interconnected 4.0 enviroments.

A supply chain attack happens when the attacker instead of targeting the main organizaton instead goes after third party vendors, suppliers or software providers that has authorized access to targets systems. In manufacturing supply chains are huge covering raw material suppliers, software vendors, logistics partners and maintenance service providers. What makes the supply chain attacks so dangerouse is that the attackers enter through a trusted party which makes it much more difficult to spot when an intrusion happens.

In a traditional, isolated manufacturing enviroment a compromised supplier might cause a delivery delay or a faulty component. In smart manufacturing however the consequenses are far more grave. Modern smart factories rely on alot of more third party vendors, each of these are a potential entry point for a hacker. It is also hard for the factory to vet the security of all these third party vendors.

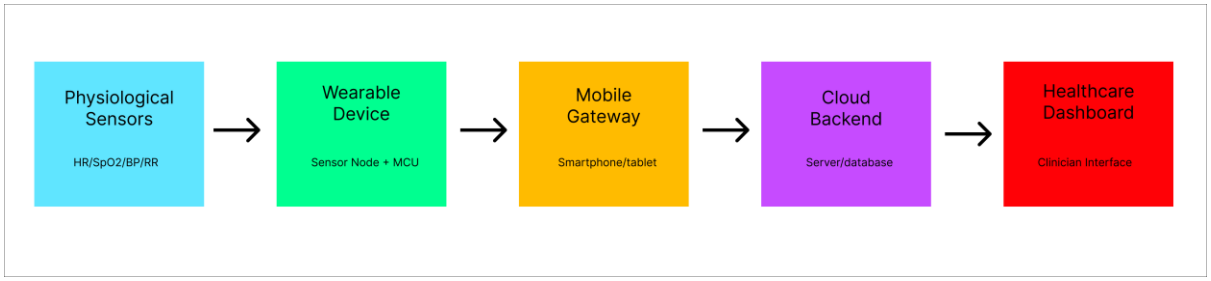
THe IT/OT convergence highlighted in Lals lecture makes this even more dangerous. A supply chain compromise that starts at the IT level can propagate laterally to the OT systems controlling the machinery. When an attack reach ISC or PLC layer the losses are not just data but stop in production, safety hazards and equipment damage(not restricted to only these). Detection is also hard because as mentioned it comes through a trusted vendor and the malware may lay dormant for a long time before activation meaning significant damage before it is discovered.

To illustrate this, consider a Norwegian automotive parts manufacturer that uses third party vendors to manage firmware updatedfor robotic wielding arms on its production line. The vendor has permanent VPN access to the manufacturers OT network. An attacker injects malicious code to the vendors update. When the vendor deploys the

update, the malicious code is installed across all robotic arms. The malware makes the arms weld bad but not bad enough to be noticed at first, the parts are shipped out to the world without the company having a clue. When the defects start getting noticed and the defects is traced back to the malware the parts are alreadu around the globe.

This example shows why supply chain attacks are so critical in smart manufacturing. The root cause is not that the company has bad security itsealf but a trusted vendor. Addressing this threat requires comprehensive third party risk management, network segmentation between IT and OT, and to use the frameworks like IEC 62443 rigiroussly.

Part B



For this task i have chose to use the Garming 8 as the typical IoT based health monitoring device. The Garmin continiously measures physiological vital signs using sensors on the underside of the watch face. IT procesces readings on an internal processor and sends the data via a BLE/wifi module to the Garmin app. The app relays the data to a cloud backend where it can be accessed through the app or web interface.

1.2

The Garmin is decomposed into 6 components.

Component 1 – Physilogical sensors(Garmin elevate gen 5 + ECG)

Hardware : Garmin Elevate Gen 5 Optical PPG sensor array (multi-wavelength LEDs + Photodetectors); separate ECG electrodes on the watch body

Function: Continiously measuers heart rate, Respitory rate.

Interfaces(Input):Optical signals reflected from the wrist cappilaries.

Interfaces(output):Raw ADC sample streams passed internallt to the processor

Component 2- sensor hub

Hardware: Ambiq Micro Apollo 3 ultra low power microcontroller

Function: Handles low-level sensor data, ADC samplign and intitial filtering

Interfaces(Input): Raw ADC streams from the Elevate Gen 5 sensor and ECG electrodes.

Interfaces(outout): Pre processed sensor data forwarded to the main processor

Component 3 – main processor

Hardware: main processor

Function: runs the garmin operating system and all health algorithms. Processes sensor h8ub data, manages display and more.

Interfaces(Input): sensor data from the Ambiq apollo 3

Interfaces(outout): processed health metrics to the display and for wifi ble transmission.

Component 4 – wireless communication

Hardware : wifi/BLE module

Function: transmits data

Interfaces(Input):data from the processor or data from the mobile gateway

Interface(output):encrypted BLE communcation to the Garmin app

Compinent 5- mobile gateway

Hardware: users smartphone(garmin app on the phone)

Function: Receives BLE data from the Garmin watch serves the data to the garmin cloud.

Interface(Input):BLE frames from the ble/wifi module; HTTPS response from the cloud backend

Interface(Output): HTTPS POST request to garming cloud

Component 6 – Cloud backend and dashboard

Hardware: Garmins cloud server infrastructure

Function: receives validates and stores health data from all users. Hosts the garmin health API.

Interfaces(Input): HTTPS request from mobile gateway

Interface(Output): Rest API responses to the Garmin ; health data exports to authorized third pary platforms such as strava.

1.3 STRIDE Threat Identification

For each of the component on Garmin watch i have identified one threat form the STRIDE framework.

- Physilogical sensors(Garmin elevate gen 5 + ECG)

Spoofing: Impersonating sensor by injecting false signals

Tampering: modifying how readings are sampled

Repudation: no secure audot logs for sensor connectivity

Information Disclosure:Raw biometric data exposed to the internal bus
DoS: excessive read commands will stop the sensor
Elevation of privelige: exploiting sensor bus access to reach apollo 3 hub

-sensor hub

Spoofing: impersonating the sensor to send false data to main processor

Tampering: modifying firmware to alter readings

Repudation: insecure or missing logs fowarded to the processor

Information Disclosure:processed biometric data exposed to the internal bus
DoS: overloading the hub will make it not be able to get or forward readings
Elevation of privelige:firmware exploit on the sensor hub to gain acess to the main processor bus

- main processor

Spoofing: impersoneting the update server to deliver malicious firmware

Tampering: pushing a malicious firmware update via BLE MITM to alter health algorithms

Repudation: no signed log of which firmware is running or when updated

Information Disclosure: Health data in flash memory exposed is secure storage is not enforced
DoS: repeated OTA update attemps can make the processro crash or reboot.
Elevation of privelige: exploiting the bootloader to execute code with full device privelige

- wireless communication

Spoofing: fake BLE device impersonates the garmin connect app

Tampering: intercepting and altering BLE data frames

Repudation: no logs for BLE transmissions, making it hard to verify what was sent

Information Disclosure:biometric data exposed via BLE IF encryption is weak
DoS: BLE connection flood prevents the wattch to pair with the app
Elevation of privelige: exploiting BLE procol vuln to reach processor

- mobile gateway

Spoofing: fake app can impersonate real and intercept BLE data from watch

Tampering: fake app can modify data before sending to cloud

Repudation: no secure logs for data sent

Information Disclosure:unencrypted local SQLite cache
DoS: flooding the app with BLE connections prevents cloud sync
Elevation of privelige:permision exploits can grant access to health data on other apps

- Cloud backend and dashboard

Spoofing: impersonating user to send false data

Tampering: modifying stored health records or altering API responses

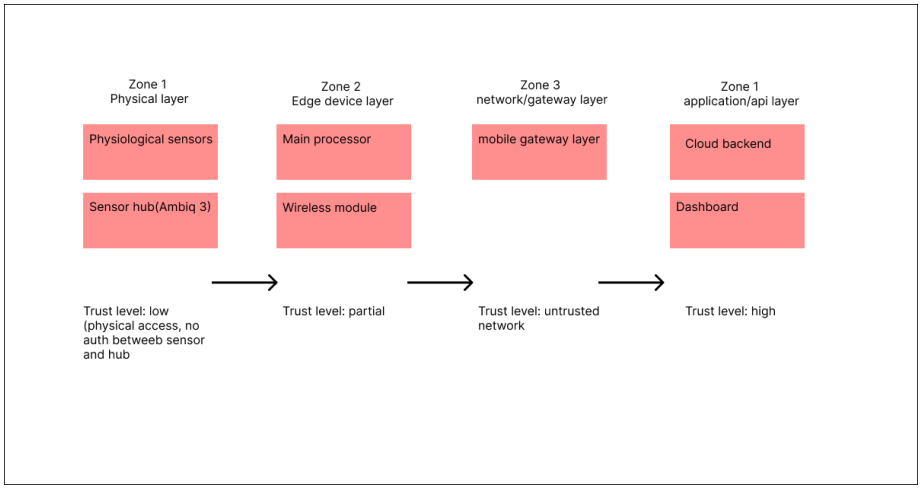
Repudation: insecure or incomplete audit logsa for API access

Information Disclosure:exposed credentials or health data via Oauth scopes given to third party apps
DoS: API request floods prevemt data uplaod from all registered users
Elevation of privelige:OAuth token theft gives attacjer admin level access to all users data

1.4

Trust boundaries surround gorups of compontents that shares the same security attributes. As Chantzis et al. (2021, p .20)explain, mapping the trust boundaries can help us understand all data flow and all areas exposed to threats. On the Garmin watch every time health data crosses a trust boundary it moves from a system it trusts to one it doesnt fully control, here is the perfect place for an attacker to strike.

Garmin is a wearable health device that collects biometric data that falls under GDPR's definition of sensitive personal data. Security at each boundary is therefore a legal obligation that must be met. With this in mind four trust zones have been identified below.



From zone 2 to 3 the BLE channel on the Garmin is the most significant trust boundary as this si the first time data travels over radio frequency meaning that anybody nearby can pick up the data, the same channel used to send data is also used for firmware updates; meaning that an attacker can intercept and inject.

The second boundary is from zone 3 to 4 where the garmin sends data to the cloud via HTTPS. Altough transport is encrypted, the cloud backend is the point where all user data meets, a breach here affects the whole platform.

B2 DREAD ANALYSIS

Two realistic attack scenarios that have basis in the STRIDE analysis i did in B1 and will be subjected to a risk evaluation using the DREAD model.

Scenario 1

This scenario is based on the tampering threat identified for the main processor The Garmin receives firmware updated through theg garmin connect app over BLE channel managed by RS9116 module. IoT devices ofted do not verify the integrity of digital signatures and firmware, making this kind of attack easiest to carry out (Chantzis et al., 2021, p, 24).
If the bootloader on the main processor does not enforce cryptographic signature verificatoin berfore accepting firmware upadates we can perdorm a MITM attack on the firmware update altering the firmware to for example give abnormal health scores.

DREAD	Score
Damage potential	9
Reproducibility	7
Exploitability Affected users	6 8
Discoverability	5
Average RIsk Score	7.0

Scenario 2

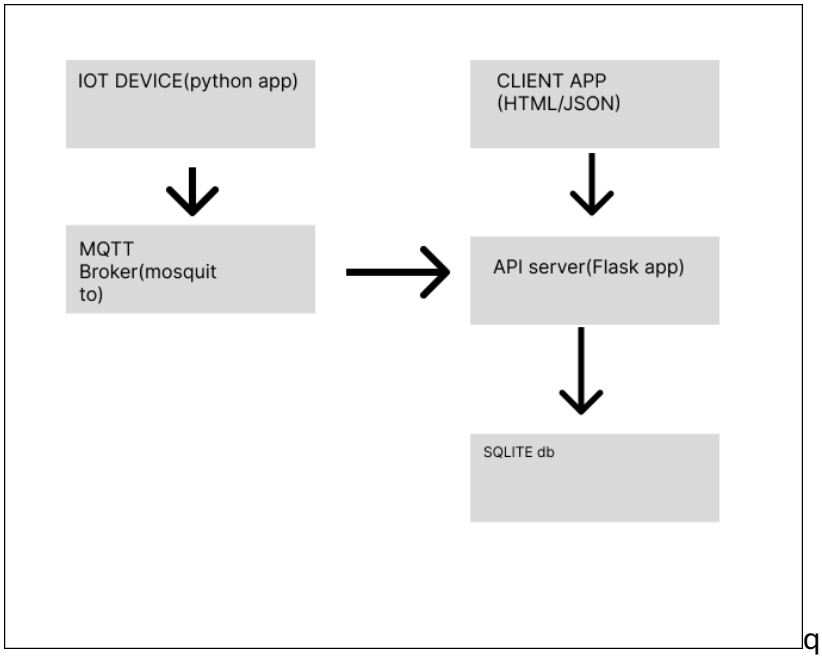
This scenatio is based on the elevation fo priveliege threath identiied in the garming connect cloud API the garmin API uses OAuth 2.0 to grant third party applications access to users heakth data, if a third party app is granted overly broad OAith permission scopes, or if tokens are stored in plaintext in the Garmin connect apps local SQLite cache on a rooted device, an attacker can get a token that allows them to call priveleged API endpoints on behalf of the user.

The danger here lies in the scale. If the attacker gets the token he/she can access all data on the cloud making the damage far greater than the one on scenario 1.

DREAD	SCORE
Damage potential	9
Reproducibility	8
Exploitability	8
Affected users	9
Discoverability	7
Average risk score	8.2

C.1

The system consists of five components communicating through a central MQTT broker. Figure C.1 shows the data flow between all components.



q

The five components and their roles are as follows:

IOT Device(Main.py): simulated sensor for soil and humidity, generates random values every 10 secs

MQTT broker : central message broker, set up with no auth for vulnerability demonstration

API server: subscribes to MQTT topic, posts readings to sql and sends data to client

Database: SQLite db stores and writes all sensor data

Client application: shows data to user, and gets data from api

C.1.2 Implementation

The entire system is containerised using Docker. Each component runs in its own container connected through an internal Docker bridge network. The MQTT is set to port 1883 and allows for anonymous connections. The sensor creates random values every 5 seconds and and sends the data to the server. Server takes the data and writes it to the db that sends it to the client interface via a api.

```python
import os
import time
import random
import paho.mqtt.client as mqtt

MQTT_BROKER = os.environ.get("MQTT_BROKER", "localhost")
MQTT_PORT = 1883
TOPIC = "sensor/humidity"

def main():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

    while True:
        try:
            client.connect(MQTT_BROKER, MQTT_PORT, 60)
            print(f"Connected to MQTT broker at {MQTT_BROKER}")
            break
        except Exception as e:
            print(f"Connection failed: {e}, retrying in 5 seconds...")
            time.sleep(5)

    client.loop_start()

    while True:
        humidity = random.randint(10, 90)
        client.publish(TOPIC, humidity)
        print(f"Published humidity: {humidity}")
        time.sleep(10)

if __name__ == "__main__":
    main()
```

```yaml
services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: mosquitto
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf
      - ./passwd:/mosquitto/config/passwd
    networks:
      - iot-net
```

```python
import os
import sqlite3
import threading
from datetime import datetime
from flask import Flask, jsonify, render_template
import paho.mqtt.client as mqtt

app = Flask(__name__)

MQTT_BROKER = os.environ.get("MQTT_BROKER", "localhost")
MQTT_PORT = 1883
TOPIC = "sensor/humidity"
DB_PATH = os.path.join(os.path.dirname(__file__), "plant.db")

def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS readings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            humidity INTEGER NOT NULL,
            time TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()

def save_reading(humidity):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    cursor.execute("INSERT INTO readings (humidity, time) VALUES (?, ?)", (humidity, timestamp)
    conn.commit()
    conn.close()
```

Authentication Required

Please sign in.

IOT — mosquitto.conf

mosquitto.conf

prosjekt > mosquitto.conf

```
listener 1883
allow_anonymous true
```

Plant Humidity Monitor

39%

Humidity	Time
39%	2026-02-20 07:58:45
53%	2026-02-20 07:58:35
38%	2026-02-20 07:58:25
85%	2026-02-20 07:58:15
66%	2026-02-20 07:58:05

Prev Page 1 of 385 Next

IOT — index.html

Open Agent Manager

index.html

prosjekt > server > templates > index.html > html > body > script > render

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Plant Humidity Monitor</title>
    <style>
        body {
            font-family: sans-serif;
            max-width: 500px;
            margin: 40px auto;
            padding: 20px;
        }
        h1 {
            margin-bottom: 20px;
        }
        .latest {
            font-size: 32px;
            margin-bottom: 20px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
        }
        th, td {
            padding: 8px;
            text-align: left;
            border: 1px solid #ccc;
        }
        th {
            background: #eee;
        }
        .pagination {
```

1.3

The following analysis maps system vulnerabilities to the OWASP Top 10 IoT Security Risks and demonstrates practical attacks for each.

#RisikoI1Weak, Guessable, or Hardcoded Passwords

-mosquittp runs with anonumous allow

I2Insecure Network Services

-MQTT on port 1883 is open and unencrypted

I3Insecure Ecosystem Interfaces

-flask API has no auth on endpoints

I7Insecure Data Transfer and Storage

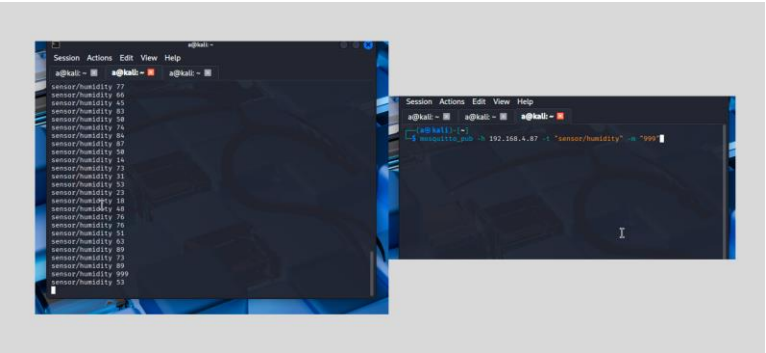
-sensor data sent in plaintext

I9Insecure Default Settings

-default mosquitto config used

Attack 1 Unauthenticated MQTT Access and Data Injection

As Chantzis et al. (2021, p. 74) note, one of the main problems with MQTT is that authentication is optional, and even if used, it is unencrypted by default. Because the Mosquitto broker is configured with allow_anonymous true, any device on the network can connect, subscribe to all topics, and publish arbitrary data without credentials.



Attack 2 – Plaintext Traffic Capture with tcpdump

Because MQTT traffic on port 1883 is unencrypted, a man-in-the-middle attacker on the same network segment can capture all messages including credentials transmitted in the CONNECT packet. Chantzis et al. (2021, p. 74) show that the CONNECT packet stores username and password as cleartext, visible in any packet capture.

look at screenshow above.

1.4

Security Measures

Enable MQTT Authentication , Enable TLS on MQTT, **Implement Access Control Lists, Protect the Flask API, Harden Docker Configuration**

1.5 Conclusion

The plant monitoring solution demontrates a perfect app to show insecurities in simple solutions and how easy they can be to mitigate when tested and exposed.

C.2

The project selected for this task is: MP3 player control with webserver using ESP32 WIFI from https://projecthub.arduino.cc/carlosvolt/mp3-player-control-with-webserver-using-esp32-wifi-268718

The setup() handles initialization tasks like connecting to Wi-Fi and starting the web server. The loop() is empty because the web server runs automatically in the background.

The project will be analyzed using MITRE EMB3D frameworj

Device Properties:

PID-11: The ESP32 is the main processor that runs the firmware and handles all communication.
PID-14: The board has a USB port that allows anyone with physical access to read or modify the firmware.
PID-211: The web server uses plain HTTP, meaning all traffic is sent without encryption.
PID-321: The Wi-Fi password is written directly into the source code, making it easy to find.
PID-37: There is no login or authentication on any of the control endpoints.

Threats:

TID-406: Anyone on the same Wi-Fi network can control the device freely, since there is no authentication.
TID-408: Because HTTP is unencrypted, anyone on the network can read the traffic.
TID-329: The Wi-Fi password is stored in plain text and can be found by reading the source code or the device's memory.
TID-115: Physical access to the USB port is enough to extract the firmware and any stored credentials.
TID-405: The device has no protection against being flooded with requests, which could make it stop responding.

Mitigations:

MID-018: Add a login system so only authorized users can send commands to the device.
MID-036: Switch to HTTPS so that communication between the browser and the device is encrypted.
MID-038: Store the Wi-Fi password securely on the device instead of writing it into the code.
MID-030: Enable flash encryption so that even if someone reads the device memory, they cannot understand its contents.
MID-016: Disable features that are not being used, such as the debug port and Bluetooth, to reduce risk.

MITRE EMB3D threat model is a useful framework for finding weaknesses in IoT devices. Using it in this projects shows that several common security problems exist, such as no auth, unencrypted communications, and passwords stored in plaintext. All of these can be fixed easily.

REFERANSELISTE

Chantzis, F., Stais, I., Calderon, P., Deirmentzoglou, E., & Woods, B. (2021). *Practical IoT hacking: The definitive guide to attacking the Internet of Things*. No Starch Press.

Docker Inc. (n.d.). *Docker documentation*. Retrieved from https://docs.docker.com/

Eclipse Foundation. (n.d.). *Eclipse Mosquitto: An open source MQTT broker*. Retrieved from https://mosquitto.org/

Fidan, F. (2025, April). *Garmin Fenix 8 review – internals teardown*. F Tipi Blog. Retrieved from https://www.f-blog.info/garmin-fenix-8-review-but-only-the-internals/

Flask. (n.d.). *Flask web framework documentation*. Retrieved from https://flask.palletsprojects.com/

Garmin International. (2024). *fēnix 8 Series Owner's Manual*. Retrieved from https://www8.garmin.com/manuals/webhelp/GUID-EECCAC99-90D6-4AB1-9A3A-EC433D3365E2/EN-US/

Garmin International. (n.d.). *Garmin Health API*. Retrieved from https://developer.garmin.com/health-api/overview/

MITRE. (2024). *EMB3D threat model framework*. Retrieved from https://emb3d.mitre.org/

MySQL AB. (n.d.). *MySQL 8.0 reference manual*. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/

Nmap Project. (n.d.). *Ncrack – Network authentication cracking tool*. Retrieved from https://nmap.org/ncrack/

NotebookCheck. (2025, April). *Garmin Fenix 8 Solar: Smartwatch teardown reveals old technology*. Retrieved from https://www.notebookcheck.net/

OWASP Foundation. (n.d.). *OWASP Internet of Things Project – Top 10 IoT Vulnerabilities*. Retrieved from https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project

Pallaoro, A. (2023). *Paho MQTT Python client*. Eclipse Foundation. Retrieved from https://pypi.org/project/paho-mqtt/